

Homework 4 Part 1 - Solutions

This is an individual assignment.

Write your answers using markdown cells or embed handwritten answers with `IPython.display.Image`.

Exercise 1 (8 points)

Consider the following application scenarios. Between the Linear Support Vector Machines (SVM) and Logistic Regression (LR), which one would you select for each scenario? Justify your answers.

1. (2 points) **Detecting fraudulent transactions in a credit card dataset, where the fraudulent transactions might be a minority class and are likely to be distinct from normal transactions.**

The fact that classes are likely to not be normal, this makes it disadvantageous for LR since it assumes class Gaussianity. Moreover, in a class imbalance scenario, SVMs are often preferred as their aim is to find the optimal hyperplane that maximizes the margin between classes. This characteristic makes SVM more robust to outliers and capable of handling separable datasets better than logistic regression. In situations where there are outliers present, SVM might provide better classification performance.

2. (2 points) **Text classification tasks, where each word or n-gram can be considered as a feature, leading to high-dimensional feature spaces.**

Linear SVM is known for its effectiveness in high-dimensional spaces, especially when the number of features is greater than the number of samples. The SVM decision boundary is determined by a subset of training samples, known as support vectors, making it computationally efficient and well-suited for high-dimensional data.

3. (2 points) **Medical diagnosis, where the occurrence of a rare disease is much lower than that of the healthy population.**

Linear SVM can handle imbalanced datasets well, where the number of instances in one class significantly outweighs the other. SVM can optimize the margin by considering support vectors from both classes, resulting in a balanced decision boundary.

4. (2 points) **Large-scale text categorization, where the focus is on assigning documents to predefined categories, rather than estimating the probability of belonging to a specific class.**

Logistic Regression outputs probabilities of class membership, which might be computationally expensive for large datasets. On the other hand, SVM solely relies on the decision boundary, making it more efficient when probability estimation is not required.

Exercise 2 (10 points)

Consider a feed-forward multilayer perceptron with a D - M - D architecture ($M \leq D$) and linear activation functions. Show that such a neural network, when trained to predict the input at the output layer, performs principal component analysis by considering the minimization it solves. Indicate if the network will require bias terms.

The network is trained to minimize the mean squared error (MSE) objective function between the predicted output and the actual input during the training process.

The reconstructed input (at the output layer) can be defined as:

$$\hat{x} = \sum_{i=1}^M y_i a_i$$

And, the hidden layer projection can be defined as:

$$y_j = x^T a_j$$

where $A^T A = I$.

The residual error is thus:

$$\epsilon = x - \hat{x} = \sum_{i=M+1}^D y_i a_i$$

The MSE objective function can be rewritten as:

$$\begin{aligned}
J &= E \left[\|\epsilon\|_2^2 \right] \\
&= E \left[\left(\sum_{i=m+1}^D y_i a_i \right) \left(\sum_{i=m+1}^D y_i a_i \right) \right] \\
&= \sum_{j=m+1}^D E[y_j^2], \text{ because } a_i^T a_j = 0, \forall i \neq j \text{ and } a_i^T a_j = 1, \forall i = j \\
&= \sum_{j=m+1}^D E[(a_j^T \mathbf{x})(\mathbf{x}^T a_j)] \\
&= \sum_{j=m+1}^D a_j^T E[\mathbf{x}\mathbf{x}^T] a_j \\
&= \sum_{j=m+1}^D a_j^T R_x a_j
\end{aligned}$$

Minimizing the objective and incorporate Lagrange parameters for $A^T A = I$:

$$\begin{aligned}
\frac{\partial J}{\partial a_j} &= 2(R_x a_j - \lambda_j a_j) = 0 \\
R_x a_j &= \lambda_j a_j
\end{aligned}$$

So, the sum of the error is the sum of the eigenvalues of the unused eigenvectors. So, we want to select the eigenvectors with the m largest eigenvalues. Similar to PCA.

Exercise 3 (6 points)

Compare and contrast the objective functions of PCA and Fisher Linear Discriminant Analysis (FLDA) for dimensionality reduction. Specifically, explain how FLDA incorporates class information that PCA ignores, defining all terms such as within-class scatter and between-class scatter matrices. And, discuss the advantages and disadvantages of each method, particularly in the context of supervised vs unsupervised learning.

Principal Component Analysis (PCA) seeks projection directions $\mathbf{w}$ that maximize variance in the projected space. Its objective function is:

$$\arg_{\mathbf{w}} \max \mathbf{w}^T R_X \mathbf{w} \quad \text{subject to } \|\mathbf{w}\| = 1$$

where R_X is the data covariance matrix of the de-meaned feature matrix. PCA finds eigenvectors of R_X corresponding to the largest eigenvalues, effectively preserving the directions of maximum data spread. Critically, PCA is unsupervised because it treats all data points identically regardless of their class membership.

Fisher Linear Discriminant Analysis (FLDA), in contrast, explicitly uses class labels to find projections that maximize class separability. Its objective function is:

$$\arg_{\mathbf{w}} \max \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$$

This ratio, called the Fisher criterion, balances two competing goals: maximizing between-class separation (numerator) while minimizing within-class scatter (denominator).

The key *innovation* of FLDA lies in decomposing data scatter into two components based on class structure:

1. Within-Class Scatter Matrix (S_W) measures how spread out data points are around their respective class means: $S_W = \sum_c \sum_{i \in \text{class } c} (\mathbf{x}_i - \mu_c)(\mathbf{x}_i - \mu_c)^T$, where μ_c is the mean of class c . This captures the "compactness" of each class; ideally, we want this to be small so that instances of the same class cluster tightly together.
2. Between-Class Scatter Matrix (S_B) measures how separated the class means are from the global mean: $S_B = \sum_c n_c (\mu_c - \mu)(\mu_c - \mu)^T$, where n_c is the number of samples in class c and μ is the overall data mean. This captures class separation; ideally, we want this to be large so that different classes are far apart.

The fundamental insight is that total scatter $S_T = S_W + S_B$. While **PCA maximizes S_T without considering how it's distributed**, FLDA specifically maximizes the ratio S_B/S_W , directly optimizing for the supervised learning goal of class discrimination.

PCA's strengths stem from its generality. Being unsupervised, it works when class labels are unavailable or unreliable. It's optimal for data reconstruction and compression, minimizing information loss as measured by variance. PCA can project to any desired number of dimensions and involves solving a standard eigenvalue problem, making it computationally straightforward.

However, PCA's limitations become apparent in classification tasks. Directions of high variance may not separate classes well. PCA can project classes onto overlapping regions even when they're separable in the original space.

FLDA's strengths are precisely in supervised scenarios. By explicitly optimizing class separability, it often achieves better classification performance with fewer dimensions. For C classes, FLDA can find at most $C-1$ discriminative directions, which is often sufficient. It focuses on the discriminative structure, making it robust to high-variance but irrelevant features that might mislead PCA.

Yet FLDA's limitations are significant. It requires labeled training data, making it inapplicable for exploratory analysis or unsupervised settings. The $C-1$ dimension limit can be restrictive for visualization or when more dimensions are needed. FLDA assumes

approximately Gaussian class distributions and linear separability - it struggles with complex, non-linear class boundaries. The within-class scatter matrix can become singular when sample size is small relative to dimensionality ($n < d$), causing numerical instability. FLDA is also more sensitive to outliers since they disproportionately affect class scatter estimates.

Exercise 4 (10 points)

Design the simplest feed-forward MLP, by specifying numerical values for its

weights and biases, using the threshold activation function, $\phi(x) = \begin{cases} 1 & x > 0 \\ -1 & x \leq 0 \end{cases}$,

to solve the following scenarios:

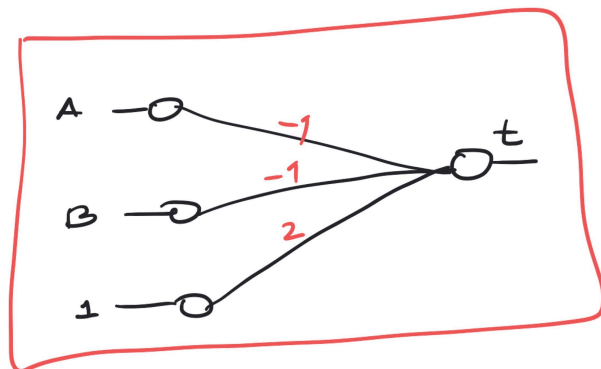
- (2 points) The NAND logic gate, $\overline{A \cap B} = \overline{A} \cup \overline{B}$, where $x_i \in \{0, 1\}, i = 1, 2$.

```
In [1]: from IPython.display import Image
Image('figures/nand_gate.jpg', width=800)
```

Out[1]: NAND

$$t = \overline{A \cap B} = \overline{A} \cup \overline{B} \equiv \text{NOT } A \text{ OR NOT } B$$

A	B	$\overline{A \cap B}$
0	0	1
0	1	1
1	0	1
1	1	0



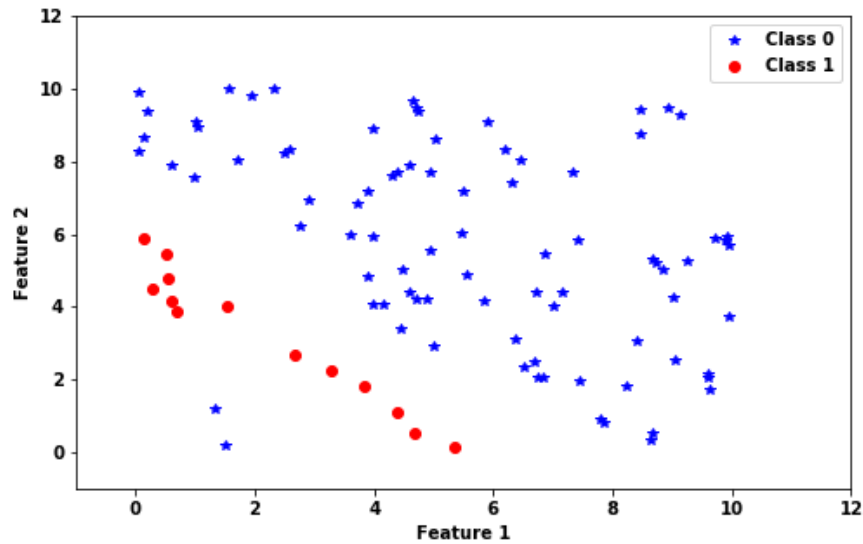
$$t = \phi(-A - B + 2)$$

$$\text{where } \phi(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

- (4 points) The following two-dimensional dataset,

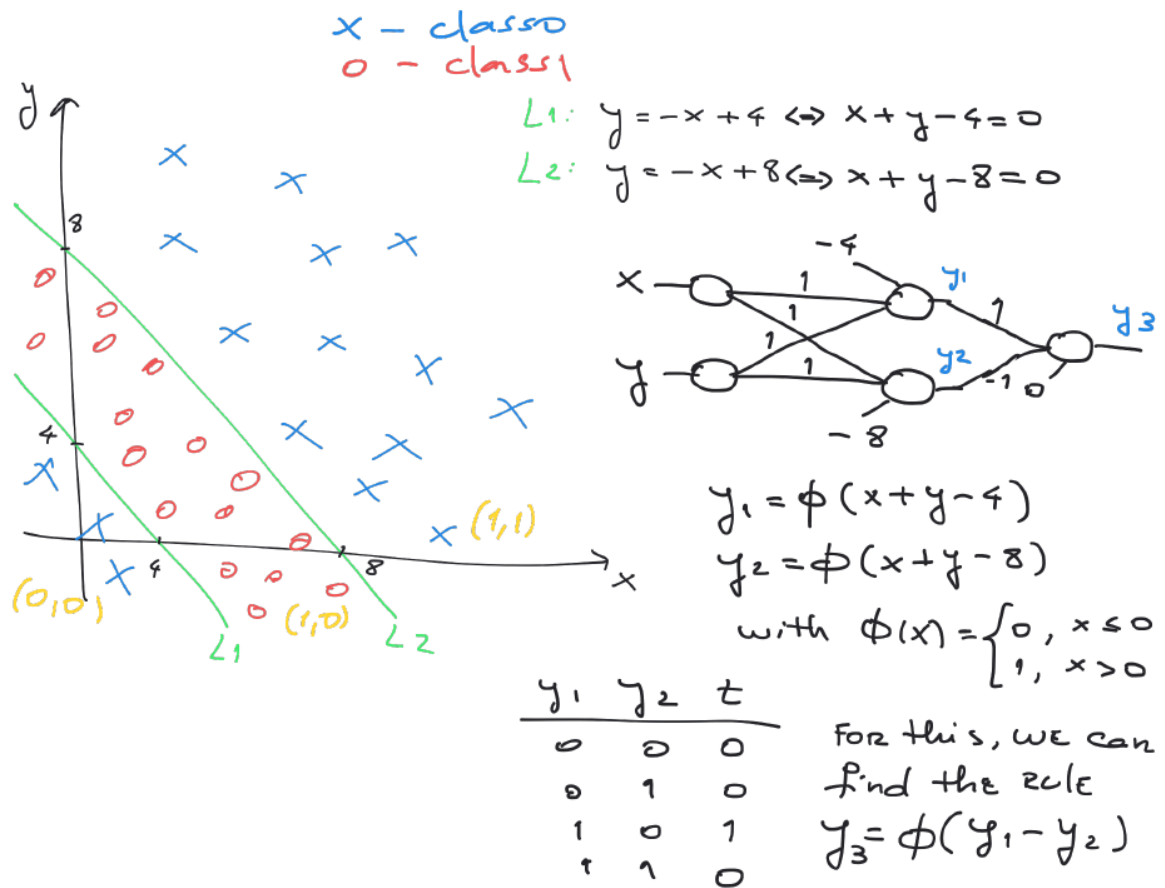
```
In [2]: from IPython.display import Image
Image('figures/dataset.png', width=500)
```

Out[2]:



In [3]: `Image('figures/Q4solution.png',width=800)`

Out[3]:



3. (4 points) The given ground truth table,

x1	x2	x3	t
0	0	0	0

x1	x2	x3	t
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

```
In [4]: import matplotlib.pyplot as plt
%matplotlib inline
plt.style.use('bmh')
import numpy as np

X = np.array([[0,0,0],
              [0,0,1],
              [0,1,0],
              [0,1,1],
              [1,0,0],
              [1,0,1],
              [1,1,0],
              [1,1,1]]).T

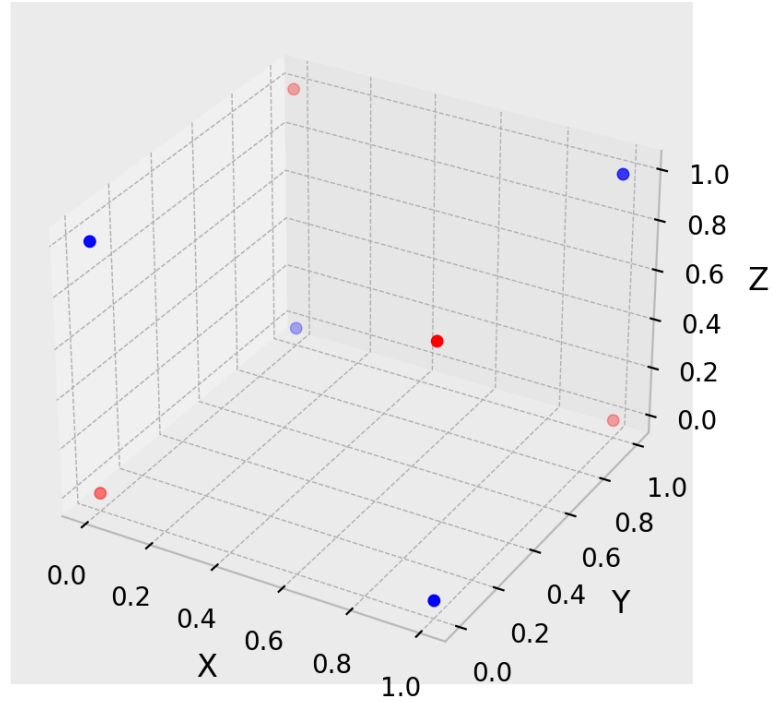
t = np.array([0,1,1,0,1,0,0,1])

X.shape, t.shape
```

Out[4]: ((3, 8), (8,))

```
In [5]: %matplotlib notebook

ax = plt.figure().add_subplot(projection='3d')
ax.scatter(X[0,t==0], X[1,t==0], X[2,t==0], c='red')
ax.scatter(X[0,t==1], X[1,t==1], X[2,t==1], c='blue')
ax.set_xlabel('X'); ax.set_ylabel('Y'); ax.set_zlabel('Z');
```



The mapper function can be written as,

$$\begin{aligned}
 y(\mathbf{x}) &= \bar{x}_1 \bar{x}_2 x_3 + \bar{x}_1 x_2 \bar{x}_3 + x_1 \bar{x}_2 \bar{x}_3 + x_1 x_2 x_3 \\
 &= \bar{x}_1 (\bar{x}_2 x_3 + x_2 \bar{x}_3) + x_1 (\bar{x}_2 \bar{x}_3 + x_2 x_3) \\
 &= \bar{x}_1 (x_2 \oplus x_3) + x_1 (\overline{x_2 \oplus x_3})
 \end{aligned}$$

Alternative Solution (written by TA Andrew Jensen, Summer C 2023)

I did this problem by plotting each of the 8 points at the corners of a cube, then creating 3 separating planes to dissect the cube into sub-regions. The first layer defines the plane geometry, the second layer determines which region of the cube each point is in, and the last layer decides the class based on the region of the cube.

$$W_1 = \begin{bmatrix} -1 & -1 & 0 \\ -1 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{and} \quad b_1 = \left[\frac{1}{2} \quad -\frac{1}{2} \quad -\frac{1}{2} \right]^T$$

The second layer is

$$W_2 = \begin{bmatrix} -1 & 1 & -1 \\ 1 & -1 & 1 \end{bmatrix} \quad \text{and} \quad b_2 = \left[\frac{1}{2} \quad -\frac{3}{2} \right]^T$$

The last layer is:

$$W_3 = \begin{bmatrix} 1 & 1 \end{bmatrix} \quad \text{and} \quad b_3 = 0$$

The overall equation for the perceptron is:

$$y(\mathbf{x}) = \phi(W_3\phi(W_2\phi(W_1\mathbf{x} + b_1) + b_2) + b_3)$$

Exercise 5 (4 points)

Explain why the input-to-hidden weights must be different from each other (e.g., random) or else learning cannot proceed well. Specifically, what happens if the weights are initialized so as to have identical values?

Suppose the input to hidden weights are set equal to the same value, say w_o , then $w_{ij} = w_o$. Then we have

$$v_j = \phi(v_i) = \sum_{j=1}^D w_{ji}\mathbf{x}_i = w_o \sum_{j=1}^D \mathbf{x}_i = w_o\mathbf{x}$$

This means that $y_j = \phi(v_j)$ is constant, say y_o . Clearly, whatever the topology of the network, setting the w_{ji} to be a constant is equivalent to changing the topology so that there is only a single input unit, whose input to the next layer is $\mathbf{x}_o$. As a result of this objective of one-layer and number of input units in the next layer, the network will not train well.

Exercise 5 (7 points)

Answer the following questions, and provide appropriate justifications, about Convolutional Neural Networks (CNNs):

1. (2 points) **Would you prefer to add more filters in the first convolutional layer or the second convolutional layer?**

Given the choice, I would prefer to have more filters in the second convolutional layer because the first convolutional layer is known for learning low-level features, which there are so many. But there are a lot more ways of arranging low-level features (in the second layer).

2. (2 points) **Why would you want to add a max pooling layer rather than a convolutional layer with the same stride?**

A max pooling layer has no parameters at all, whereas a convolutional layer has quite a few (see the previous questions).

3. (3 points) **Consider a CNN composed of three convolutional layers, each with 3×3 kernels, a stride of 2, and "same" padding. The lowest layer outputs 100 feature maps, the middle one outputs 200, and the top one outputs 400. The input images are RGB images of 200×300 pixels. What is the total number of parameters in the CNN?**

Since its first convolutional layer has 3×3 kernels, and the input has three channels (red, green, and blue), each feature map has $3 \times 3 \times 3$ weights, plus a bias term. That's 28 parameters per feature map. Since this first convolutional layer has 100 feature maps, it has a total of 2,800 parameters. The second convolutional layer has 3×3 kernels and its input is the set of 100 feature maps of the previous layer, so each feature map has $3 \times 3 \times 100 = 900$ weights, plus a bias term. Since it has 200 feature maps, this layer has $901 \times 200 = 180,200$ parameters. Finally, the third and last convolutional layer also has 3×3 kernels, and its input is the set of 200 feature maps of the previous layers, so each feature map has $3 \times 3 \times 200 = 1,800$ weights, plus a bias term. Since it has 400 feature maps, this layer has a total of $1,801 \times 400 = 720,400$ parameters. All in all, the CNN has $2,800 + 180,200 + 720,400 = 903,400$ parameters.

On-Time (5 points)

Submit your assignment before the deadline.

Submit Your Solution

Confirm that you've successfully completed the assignment.

Along with the Notebook, include a PDF of the notebook with your solutions.

`add` and `commit` the final version of your work, and `push` your code to your GitHub repository.

Submit the URL of your GitHub Repository as your assignment submission on Canvas.
